Data Management for Data Publishers

Table of Contents

Chapter 1. Introduction

1.1. Introduction

1.1.1. Purpose and Scope

This documentation provides comprehensive guidance for data publishers, metadata managers, and data stewards who want to create and manage data catalogs using the simple_data_catalog framework. This guide covers the fundamentals of metadata management for data cataloging, with an emphasis on generic metadata management based on W3C standards.

The goal of this documentation is to both train people in the basics of (meta)data management and the use of the simple_data_catalog framework. It serves as both an educational resource for understanding metadata standards and a practical guide for implementing data catalogs in real-world scenarios.

1.1.2. Overview of Simple Data Catalog

simple_data_catalog is a LinkML-based data catalog management tool that provides a low-barrier-to-entry approach for creating DCAT-compatible data catalogs. The framework is built around four core metadata areas:

- **Data Catalog Metadata** - Based on DCAT 3 for describing datasets, distributions, and data services
- **Data Lineage** - Based on PROV-O for tracking data provenance and transformation history
- **Data Quality** - Based on DQV for measuring and reporting data quality metrics
- **Data Policy** - Based on ODRL 2.2 for expressing usage rights and permissions

The framework uses a YAML-based configuration that generates RDF graphs compliant with these standards, making it easy to create interoperable data catalogs without requiring deep technical expertise.

1.1.3. Target Audience

This guide is intended for:

- **Data Publishers** - Organizations and individuals who publish datasets and need to create catalog metadata
- **Metadata Managers** - Professionals responsible for managing data catalogs and metadata workflows
- **Data Stewards** - People responsible for data governance and quality management
- **Technical Staff** - IT professionals who implement and maintain catalog systems
- **Researchers** - Academics and students learning about metadata standards and data management

1.1.4. Prerequisites

Readers should have:

- Basic understanding of data concepts and metadata
- Familiarity with web technologies and data formats
- Experience with data management workflows
- Interest in standards-based approaches to data cataloging

1.1.5. How to Use This Guide

This documentation is structured to support both learning and practical implementation:

- **Progressive Complexity** - Each chapter starts with theoretical foundations and builds to advanced real-world examples
- **Standards-Based** - All implementations are grounded in W3C standards (DCAT, PROV, DQV, ODRL)
- **Practical Focus** - Emphasis on hands-on examples using simple_data_catalog
- **Cross-Reference** - Rich linking between chapters to show how different metadata areas work together

Each chapter includes:

- Theory and concepts behind the standard
- How the standard is implemented in simple_data_catalog
- Progressive examples from basic to advanced
- Best practices and common pitfalls

1.2. Document Structure

This guide is organized into chapters covering each metadata area:

1. **Data Catalog Metadata (DCAT)** - Core catalog concepts and dataset description
2. **Data Quality (DQV)** - Measuring and improving data quality
3. **Data Lineage (PROV)** - Tracking data provenance and transformations
4. **Data Policy (ODRL)** - Managing usage rights and permissions

1.3. Documentation Chapters

1.3.1. 1. Data Catalog Metadata (DCAT)

Learn the fundamentals of data cataloging using DCAT 3 standards.

Topics covered: * DCAT 3 core concepts and vocabulary * Creating datasets and distributions *

Catalog organization and structure * Best practices for metadata completeness

1.3.2. □ 2. Data Quality (DQV)

Implement data quality measurement and reporting using DQV.

Topics covered: * DQV quality metrics and measurements * Quality assessment workflows * Quality annotation and reporting * Continuous quality improvement

1.3.3. □ 3. Data Lineage (PROV)

Master data provenance tracking using PROV-O standards.

Topics covered: * PROV-O entities, activities, and agents * Recording data transformations * Building lineage graphs * Provenance documentation strategies

1.3.4. □□ 4. Data Policy (ODRL)

Manage usage rights and permissions using ODRL 2.2.

Topics covered: * ODRL policies and permissions * Rights expression and licensing * Usage constraint management * Policy implementation strategies

1.4. □ Learning Path

This guide is designed for progressive learning:

1. **Beginner** → Start with this Introduction for foundational concepts
2. **Intermediate** → Progress through DCAT and DQV for core cataloging skills
3. **Advanced** → Explore PROV and ODRL for comprehensive catalog management

1.5. □ Related Resources

1.5.1. Framework Documentation

- simple_data_catalog - Main data catalog tool
- simple_data_catalog_model - Data model definitions

1.5.2. W3C Standards

- DCAT 3 Specification - Data Catalog Vocabulary
- PROV-O Specification - Provenance Ontology
- DQV Specification - Data Quality Vocabulary
- ODRL 2.2 Specification - Open Digital Rights Language

1.6. How to Navigate

- **Sequential Reading** - Follow chapters in order for comprehensive learning
- **Topic-Based** - Jump directly to chapters relevant to your immediate needs
- **Cross-Reference** - Use internal links to explore related concepts across chapters

---

Ready to get started? Continue with the Data Catalog Metadata chapter to begin building your data catalog management skills.

1.7. Finally

If you found this website/book useful, please consider supporting its development through the sponsorship button in the top right corner of the page.

Chapter 2. Getting Started

This section helps you get up and running with the simple_data_catalog framework quickly.

2.1. Prerequisites

Before you begin, ensure you have the following installed:

- **Python 3.8+** - Required for the framework
- **Git** - For version control and template management
- **Copier** - Template tool for creating catalogs

2.2. Installation Steps

2.2.1. Step 1: Install Python

Ensure you have Python 3.8 or later installed:

```
python --version
# Should show Python 3.8.x or later
```

2.2.2. Step 2: Install Copier

Install Copier using pip:

```
pip install copier
```

2.2.3. Step 3: Clone the Simple Data Catalog Template

Get the template repository:

```
git clone https://github.com/uuidea/simple_data_catalog.git
cd simple_data_catalog
```

2.3. Creating Your First Catalog

2.3.1. Step 1: Generate Catalog Structure

Use Copier to create a new catalog from the template:

```
copier copy gh:uuidea/simple_data_catalog my-data-catalog
```

2.3.2. Step 2: Navigate to Your Catalog

```
cd my-data-catalog
```

2.3.3. Step 3: Basic Catalog Structure

Your new catalog will have this structure:

```
my-data-catalog/
├──── catalog.yaml          # Main catalog configuration
├──── datasets/              # Dataset descriptions
│     ├──── dataset1.yaml
│     └──── dataset2.yaml
├──── examples/              # Example configurations
└──── README.md
```

2.3.4. Step 4: Configure Your Catalog

Edit the main `catalog.yaml` file:

```yaml
# catalog.yaml
dcat:Catalog:
  dcterms:title: "My Data Catalog"
  dcterms:description: "A comprehensive data catalog"
  dcterms:publisher:
    foaf:name: "Your Organization"
    foaf:mbox: "contact@yourorg.org"
  dcat:dataset: []
```

2.3.5. Step 5: Add Your First Dataset

Create a simple dataset in `datasets/example.yaml`:

```yaml
# datasets/example.yaml
dcat:Dataset:
  dcterms:title: "Example Dataset"
  dcterms:description: "A sample dataset to demonstrate the catalog"
  dcterms:identifier: "example-dataset-001"
  dcat:distribution:
    - dcat:Distribution:
        dcterms:title: "CSV Download"
        dcterms:format: "text/csv"
        dcat:accessURL: "https://example.org/data.csv"
```

2.3.6. Step 6: Update Catalog to Include Dataset

Modify `catalog.yaml` to reference your dataset:

```
dcat:Catalog:
  dcterms:title: "My Data Catalog"
  dcterms:description: "A comprehensive data catalog"
  dcterms:publisher:
    foaf:name: "Your Organization"
    foaf:mbox: "contact@yourorg.org"
  dcat:dataset:
    - !include datasets/example.yaml
```

2.4. Generating Your Catalog

2.4.1. Step 1: Install Dependencies

```
pip install -r requirements.txt
```

2.4.2. Step 2: Generate RDF Output

```
python generate_catalog.py
```

This will generate: * `catalog.ttl` - RDF/Turtle format * `catalog.jsonld` - JSON-LD format * `catalog.html` - Human-readable HTML view

2.4.3. Step 3: View Your Catalog

Open the generated HTML file in your browser:

```
# If you have a local web server
python -m http.server 8000
# Then visit http://localhost:8000/catalog.html
```

2.5. Next Steps

Congratulations! You now have a basic working data catalog. Continue with these chapters to expand your catalog:

- Data Catalog Metadata (DCAT) - Learn comprehensive cataloging
- Data Quality (DQV) - Add quality metrics
- Data Lineage (PROV) - Track data provenance

- Data Policy (ODRL) - Manage usage rights

2.6. Troubleshooting

2.6.1. Common Issues

Issue: Python version not supported **Solution:** Install Python 3.8+ from https://python.org/downloads/

Issue: Copier command not found **Solution:** Ensure pip install copier completed successfully and your PATH includes pip packages

Issue: Template generation fails **Solution:** Check your internet connection and ensure the GitHub repository is accessible

Issue: RDF generation errors **Solution:** Validate your YAML syntax and ensure all required fields are present

2.6.2. Getting Help

- Check the main repository for documentation
- Review the data model specification
- Open an issue on GitHub for specific problems

Chapter 3. Data Catalog Metadata (DCAT)

3.1. 1.1 Introduction to Data Cataloging

Data cataloging is the systematic practice of describing your data resources so that others can discover, understand, and use them effectively. Think of it as creating a comprehensive library catalog for your datasets - without proper descriptions, even the most valuable data remains hidden and unusable to potential users. In today's data-driven organizations, data catalogs have evolved from simple inventories into strategic tools that enable effective data management, support governance frameworks, and drive business value through better data utilization.

3.1.1. The Role of Data Catalogs in Modern Organizations

In today's world, we collect and use more data than ever before. This data comes from all sorts of places - customer interactions, business operations, sensors, and more. The problem is, with so much data spread across different systems and teams, it's easy to lose track of what you have, where it is, and who's responsible for it.

Data catalogs solve this problem by creating a shared place to document all your data. Think of it like a library catalog - instead of books, it tracks your datasets. This makes it much easier for everyone in the organization to find the data they need when they need it.

Beyond just organization, data catalogs help with the bigger picture of data management. They create a shared understanding across the company about what data exists and how it should be handled. This is crucial for maintaining data quality, tracking how data moves through different systems (what we call "lineage"), and making sure everyone follows the same rules about data access and privacy.

For data creators and publishers specifically, catalogs provide a way to showcase your work and make sure it gets used properly. When you document your datasets in the catalog, you're not just listing them - you're explaining their value, their quality, and how others can use them effectively. This makes your work more visible and helps build your reputation as a reliable data source within the organization. it establishes you as the official source for that particular dataset.

In short, data catalogs aren't just about finding data - they're about making sure the right data gets to the right people at the right time, while maintaining all the standards and safeguards that good data management requires.

3.1.2. Data Catalogs and the FAIR Principles

The FAIR principles - Findable, Accessible, Interoperable, and Reusable - are simple rules for good data management. They help make sure data is useful and easy to work with. Data catalogs are an important tool to achieve this.

When we talk about **Findable**, we mean making data easy to locate. A good data catalog does this by providing detailed descriptions of each dataset, using standard ways to identify data, and organizing it so you can search through it. Think of it like a library catalog that helps you find books without having to search every shelf.

For **Accessible**, the catalog tells you not just where the data is, but how to get to it. It explains what steps you need to take, who to contact, and any rules you need to follow. This way, you know exactly what's required to use the data, without running into unexpected roadblocks.

Interoperability means making sure different systems can understand and work with the data. The catalog helps with this by documenting the data's structure, the standards it follows, and how it relates to other datasets. It's like having a translator that helps different computer systems talk to each other.

Finally, **Reusable** is about making sure data can be used again and again. The catalog does this by recording where the data came from, how good it is, what permissions are needed to use it, and how to properly credit the people who created it. This way, others can trust the data and use it responsibly in their own work.

3.1.3. DCAT: The Standard That Makes Cataloging Work

The Data Catalog Vocabulary (DCAT) is a W3C standard that provides a common language for describing datasets. By using DCAT, your data catalog becomes:

- **Interoperable**: Other systems can understand and process your metadata
- **Searchable**: Your data can be found through federated search across multiple catalogs
- **Future-proof**: Your catalog follows established best practices that won't become obsolete

Just as with the other standards referenced in this work, the DCAT model can also be seen as an anatomy of (a) data catalog(s). It helps us think about what we might need to know about datasets, services and distributions, and in doing so, helps us think about and scope the actions we should take to obtain and share this information.

Main DCAT Elements Used in This Document

```
classDiagram
    class Resource {
        identifier*
        title
        description
        publisher
        contactPoint
        license
        theme[]
        version
        status
        issued
        modified
    }
    class Dataset {
        temporal
        inSeries
        distribution[]
    }
    class DataCatalog {
```

```
        dataset[]
    }
    class DatasetSeries

    class Distribution {
        identifier*
        title
        description
        accessURL
        format
        version
        issued
        modified
    }

    class DataService {
        endpointURL
        servesDataset[]
    }

    class Concept {
        identifier*
        prefLabel
        definition
        altLabel
        example
    }

    class PeriodOfTime {
        hasBeginning
        hasEnd
    }
    Resource <|-- Dataset
    Dataset <|-- DataCatalog
    Dataset <|-- DatasetSeries
    Resource <|-- DataService

    Dataset --> Distribution : distribution
    Dataset --> Concept : theme
    Dataset --> DatasetSeries : inSeries
    Dataset --> PeriodOfTime : temporal
    DataService --> Dataset : servesDataset
```

DCAT provides the building blocks for cataloging, but this chapter focuses on how to apply those building blocks to create useful, discoverable data catalogs.

3.2. 1.2 Getting Started: Your First Dataset

Let's start with the simplest possible dataset description. We'll catalog a collection of air quality measurements collected from urban monitoring stations.

3.2.1. Minimal Viable Dataset

```yaml
# dataset.yaml - Individual dataset description
- identifier: "air-quality-data"
  title: "Urban Air Quality Measurements"
  description: "Air quality index measurements from urban monitoring stations"
```

What this provides:

- `identifier`: A unique identifier that distinguishes this dataset from others
- `title`: A human-readable name that appears in search results
- `description`: A brief explanation of what the data contains

Why this matters: Even this minimal description makes your dataset discoverable. Users searching for "air quality" or "urban measurements" can now find your data and understand what it contains. This focused approach lets us concentrate on describing individual datasets effectively before we worry about organizing them into catalogs.

3.3. 1.3 Enriching Your Dataset Description

Now let's enhance our dataset description to make it more discovetherable and useful.

3.3.1. Themes: Categorizing Your Data

We'll do this by adding themes to the dataset. You can think of these as keywords or tags that help categorize the data. In DCAT the theme attribute points to somthing called a skosConcept.

Going into SKOS (Simple Knowledge Organization System) concepts in detail is beyond the scope of this chapter, but at a high level, SKOS provides a standardized way to define and manage controlled vocabularies. By using SKOS concepts for themes, we ensure that our dataset descriptions are consistent and interoperable across different catalogs and systems. For more information, see http://www.w3.org/TR/skos-reference/

First, let's define reusable concepts for our themes that we can use across multiple datasets: **New elements and what they enable:**

- `Concepts`: Reusable theme definitions that can be used across multiple datasets and even catalogs
  - `identifier`: Unique key for referencing concepts
  - `prefLabel`: Human-readable display name for the concept
  - `definition`: Clear explanation of what the concept means
- `theme`: References to concept identifiers instead of inline strings

```yaml
# concepts.yaml - Reusable theme definitions
concepts:
  - identifier: "air-quality"
    prefLabel: "Air Quality"
    definition: "The degree to which air is suitable for breathing and other uses"
  - identifier: "pollution"
    prefLabel: "Pollution"
    definition: "The introduction of contaminants into natural environment"
  - identifier: "urban"
    prefLabel: "Urban"
    definition: "Relating to or characteristic of a city or town"
  - identifier: "environmental-monitoring"
    prefLabel: "Environmental Monitoring"
    definition: "The systematic collection of environmental data"

# dataset.yaml - Enhanced dataset description
- identifier: "air-quality-data"
  title: "Urban Air Quality Measurements"
  description: "Air quality index measurements from urban monitoring stations
collected hourly"
  theme:
    - "air-quality"
    - "pollution"
    - "urban"
    - "environmental-monitoring"
  publisher:
    name: "Environmental Protection Agency"
  temporal:
    hasBeginning: "2023-01-01"
    hasEnd: "2023-12-31"
  contactPoint:
    hasEmail: "air-quality@epa.gov"
  license:
    title: "https://creativecommons.org/licenses/by/4.0/"
```

New elements and what they enable:

- `concepts`: Reusable theme definitions that can be used across multiple datasets and even catalogs
  - `identifier`: Unique key for referencing concepts
  - `prefLabel`: Human-readable display name for the concept
  - `definition`: Clear explanation of what the concept means
- `theme`: References to concept identifiers instead of inline strings

3.3.2. additional Metadata: Making Your Dataset More Useful

Now that we have added themes to categorize our dataset, let's further enrich the description with

additional metadata elements that provide more context and usability:

```
- identifier: "air-quality-data"
  title: "Urban Air Quality Measurements"
  description: "Air quality index measurements from urban monitoring stations
collected hourly"
  theme:
    - "air-quality"
    - "pollution"
    - "urban"
    - "environmental-monitoring"
  publisher:
    name: "Environmental Protection Agency"
  contactPoint:
    hasEmail: "air-quality@epa.gov"
  temporal:
    hasBeginning: "2023-01-01"
    hasEnd: "2023-12-31"
  license:
    title: "https://creativecommons.org/licenses/by/4.0/"
```

New elements and what they enable:

- `publisher`: Identifies the organization responsible for the dataset
- `contactPoint`: Uses vcard-compliant structure to provide contact information
- `temporal`: Uses dcterms:PeriodOfTime to specify the time span the data covers
- `license`: points to a licence. It is recommended to use a URL pointing to a standard licence such as Creative Commons, but a title or description can also be used.

Why these additions matter: A researcher looking for air quality data from 2023 can now quickly determine that this dataset meets their needs. They know who to contact with questions, understand their usage rights, and the concepts used for themes are properly defined and reusable across the catalog.

3.4. 1.4 From the catalog to the bookshelf: Adding Distributions

A dataset description is useful, but users need actual access to the data. Distributions describe the files and formats in which the data can be obtained. They often contain an accessURL that points to where the data can be accessed.

If the data is available through some kind of service (like an API), that would be described using a DataService element instead of a Distribution. We'll cover DataServices in a later section.

```yaml
# dataset.yaml - Dataset with distributions
- identifier: "air-quality-data"
  title: "Urban Air Quality Measurements"
  description: "Air quality index measurements from urban monitoring stations
collected hourly"
  theme:
    - "air-quality"
    - "pollution"
    - "urban"
    - "environmental-monitoring"
  publisher:
    name: "Environmental Protection Agency"
  temporal:
    hasBeginning: "2023-01-01"
    hasEnd: "2023-12-31"
  contactPoint:
    hasEmail: "air-quality@epa.gov"
  license:
    title: "https://creativecommons.org/licenses/by/4.0/"
  distribution:
    - identifier: "csv-download"
      title: "CSV Download"
      description: "Complete dataset in CSV format for analysis"
      accessURL: "https://data.epa.gov/air-quality/2023.csv"
      mediaType: "text/csv"
      byteSize: "50000000"
      format: "csv"
    - identifier: "api-access"
      title: "REST API"
      description: "Programmatic access to current and historical data"
      accessURL: "https://api.epa.gov/air-quality/v1"
      mediaType: "application/json"
      conformsTo: "https://api.epa.gov/docs/air-quality"
```

What distributions provide:

- `csv-download`: Direct file access for users who want to download and analyze data locally
  - `downloadURL`: Direct link to the file
  - `mediaType`: File format information so users know what they're getting
  - `byteSize`: File size helps users estimate download time and storage needs
  - `format`: Additional format specification
- `api-access`: Programmatic access for developers and automated systems
  - `accessURL`: Endpoint for making API requests
  - `conformsTo`: Link to API documentation so users know how to use it

Why multiple distributions matter: Different users have different needs. Researchers might want

the complete CSV file for offline analysis, while application developers prefer API access for real-time data integration. Providing multiple access methods makes your dataset useful to a broader audience. Using proper LinkML structure ensures compatibility with any compliant data catalog system.

3.5. 1.5 Managing Multiple Datasets and Services

As your data collection grows, you'll want to organize related datasets and provide services that serve multiple datasets. This is where introducing a catalog structure becomes valuable - it provides a container for managing your growing collection of related datasets and services.

Let's expand our collection to include related environmental datasets and organize them within a catalog:

```yaml
# data-catalog.yaml - LinkML compliant Container structure
concepts:
  - identifier: "air-quality"
    prefLabel: "Air Quality"
    definition: "The degree to which air is suitable for breathing and other uses"
  - identifier: "water-quality"
    prefLabel: "Water Quality"
    definition: "The chemical, physical, and biological characteristics of water"
  - identifier: "pollution"
    prefLabel: "Pollution"
    definition: "The introduction of contaminants into natural environment"
  - identifier: "environmental"
    prefLabel: "Environmental"
    definition: "Relating to natural world and impact of human activity on it"

dataCatalog:
  title: "Environmental Data Catalog"
  description: "Catalog of environmental monitoring datasets"
  publisher:
    name: "Environmental Protection Agency"
  contactPoint:
    hasEmail: "catalog@epa.gov"

datasets:
  - identifier: "air-quality-data"
    title: "Urban Air Quality Measurements"
    description: "Air quality index measurements from urban monitoring stations"
    theme:
      - "air-quality"
      - "pollution"
      - "environmental"
    temporal:
      hasBeginning: "2023-01-01"
      hasEnd: "2023-12-31"
    distribution:
```

```yaml
      - identifier: "air-quality-csv"
        title: "Air Quality CSV"
        accessURL: "https://data.epa.gov/air-quality/2023.csv"

  - identifier: "water-quality-data"
    title: "Water Quality Monitoring"
    description: "Water quality parameters from river and lake monitoring stations"
    theme:
      - "water-quality"
      - "pollution"
      - "environmental"
    temporal:
      hasBeginning: "2023-01-01"
      hasEnd: "2023-12-31"
    distribution:
      - identifier: "water-quality-csv"
        title: "Water Quality CSV"
        accessURL: "https://data.epa.gov/water-quality/2023.csv"

dataServices:
  - identifier: "environmental-api"
    title: "Environmental Data API"
    description: "Unified access to all environmental monitoring datasets"
    endpointURL: "https://api.epa.gov/environmental/v1"
    servesDataset:
      - "air-quality-data"
      - "water-quality-data"
    publisher:
      name: "Environmental Protection Agency"
  contactPoint:
    hasEmail: "api@epa.gov"
```

What this adds:

- **Multiple datasets**: Related datasets are grouped together, making it easier for users to find comprehensive environmental data
- `concepts`: Reusable theme definitions used by both datasets
- `dataServices`: A service that provides access to multiple datasets through a single interface
  - `servesDataset`: References dataset identifiers, not inline objects
  - `endpointURL`: The base URL for the API service

Why this organization matters: Users interested in environmental data can now find all related datasets in one place. Concepts are reusable across datasets, ensuring consistent theming. The unified API service makes it easy for developers to access multiple datasets without learning different interfaces for each one. This creates a coherent data ecosystem rather than isolated datasets, all following proper LinkML structure.

3.6. 1.7 Complete Example: Putting It All Together

Here's our complete environmental data catalog with all the features we've discussed:

```yaml
# data-catalog.yaml - LinkML compliant complete example
concepts:
  - identifier: "air-quality"
    prefLabel: "Air Quality"
    definition: "The degree to which air is suitable for breathing and other uses"
  - identifier: "pollution"
    prefLabel: "Pollution"
    definition: "The introduction of contaminants into natural environment"
  - identifier: "urban"
    prefLabel: "Urban"
    definition: "Relating to or characteristic of a city or town"
  - identifier: "environmental-monitoring"
    prefLabel: "Environmental Monitoring"
    definition: "The systematic collection of environmental data"
  - identifier: "public-health"
    prefLabel: "Public Health"
    definition: "The health of population as a whole"

dataCatalog:
  title: "Environmental Data Catalog"
  description: "Comprehensive catalog of environmental monitoring datasets and
services"
  publisher:
    name: "Environmental Protection Agency"
  contactPoint:
    hasEmail: "catalog@epa.gov"

datasets:
  - identifier: "air-quality-2023"
    title: "Urban Air Quality Measurements - 2023"
    description: "Air quality index measurements from urban monitoring stations
collected hourly"
    version: "2.0"
    theme:
      - "air-quality"
      - "pollution"
      - "urban"
      - "environmental-monitoring"
      - "public-health"
    publisher:
      name: "Environmental Protection Agency"
    temporal:
      hasBeginning: "2023-01-01"
      hasEnd: "2023-12-31"
    contactPoint:
      hasEmail: "air-quality@epa.gov"
```

```
    license:
      title: "https://creativecommons.org/licenses/by/4.0/"
    inSeries: "air-quality-series"

series:
  - identifier: "air-quality-series"
    title: "Urban Air Quality Time Series"
    description: "Long-term air quality monitoring program in metropolitan areas"

distributions:
  - identifier: "air-quality-csv"
    title: "Complete Air Quality Dataset (CSV)"
    description: "Full year of air quality measurements in CSV format"
    accessURL: "https://data.epa.gov/air-quality/2023.csv"
    format: "csv"

  - identifier: "air-quality-api"
    title: "Air Quality API"
    description: "Real-time and historical air quality data via REST API"
    accessURL: "https://api.epa.gov/air-quality/v1"
    conformsTo: "https://api.epa.gov/docs/air-quality"

dataServices:
  - identifier: "environmental-api"
    title: "Environmental Data API"
    description: "Unified access to all environmental monitoring datasets"
    endpointURL: "https://api.epa.gov/environmental/v1"
    servesDataset:
      - "air-quality-2023"
    publisher:
      name: "Environmental Protection Agency"
  contactPoint:
    hasEmail: "api@epa.gov"
```

This comprehensive catalog demonstrates:

- **Rich metadata** for discoverability and understanding
- **Multiple access methods** for different user needs
- **Quality and compliance** information for trust building
- **Series management** for ongoing data programs
- **Unified services** for streamlined access

3.7. 1.10 Understanding Catalogs as Datasets

Throughout this chapter, we've progressed from describing individual datasets to organizing them within catalogs. But keep in mind a catalog is itself a dataset. This concept is crucial for understanding how DCAT enables interoperability at scale.

3.7.1. The Catalog-as-Dataset Concept

In DCAT, a catalog isn't just a container - it's a first-class dataset that describes other datasets. This recursive relationship means:

```yaml
# A catalog describes itself using the same structure as any dataset
dataCatalog:
  identifier: "environmental-catalog"
  title: "Environmental Data Catalog"
  description: "A catalog of environmental monitoring datasets"
  publisher:
    name: "Environmental Protection Agency"
  contactPoint:
    hasEmail: "catalog@epa.gov"
  theme:
    - "environmental-data"
    - "government-catalog"
  license:
    title: "https://creativecommons.org/licenses/by/4.0/"
  # This catalog contains other datasets
  dataset:
    - "air-quality-data"
    - "water-quality-data"
```

3.7.2. Why This Matters

When catalogs are themselves datasets, they can be:

- **Indexed by other catalogs**: National catalogs can discover and index organizational catalogs
- **Linked across systems**: Cross-references enable federated search across multiple catalogs
- **Standardized exchange**: All catalogs use the same metadata structure regardless of content

3.8. 1.8 Best Practices for Data Cataloging

Based on our progressive example, here are the key principles for effective data cataloging:

3.8.1. Start Simple, Build Incrementally, keep it simple

It is easy to overlook how much effort and resources collecting data costs. This is no different for a data catalog. As each bit of new metadata can support new usecases and increase findability, it also increased the effort it will take to keep the data catalog up to date. The best advice is: **start simple, keep it simple**.

- Begin with basic identification (id, title, description)
- Beyond this very basic start, only add when needed:
  - Add discoverability features (keywords, publisher, temporal/spatial coverage)

    - Include access information (distributions, licensing)
    - Enhance with advanced features (versioning, quality, series)

As the data catalog gets more publishers, some will want to add more metadata than others. This should be supported, but the core should remain simple so that the barrier to entry remains low.

3.8.2. Focus on User Needs

As a publisher of data, you can think of your dataset entry as an advertisement for your data. To this end, try to keep the following user centered principles in mind:

- **Discovery**: Use descriptive titles and relevant themes
- **Understanding**: Provide clear descriptions and context. While the discriptin in the simple_data_catalog is just a simple attribute, you can actually put a fair amount of documentation in there using everything asciiidoc supports (just keep in mind that the title is the dataset itself, so you should start at the second heading level)
- **Access**: If you can, offer multiple distribution formats for different users
- **Trust**: Include publisher, contact, and quality information

3.8.3. Maintain Consistency

- Use unique, stable identifiers across your catalog
- Reuse the themes across datasets where appropriate
- Keep contact information up to date
- Follow the same metadata structure for related datasets

3.8.4. Collaboration and Governance

When creating a data catalog with multiple publishers: * Leave innitiative with the individual data publishers as much as possible ** Ensure that publishers can share the metadata they need.

- Set minimal standards for required metadata fields
    - When new information needs arrize, add data based on standards
    - establish review processes to ensure quality and consistency
    - define roles and responsibilities for catalog maintenance
    - Encourage reuse and consistency across catalog entries.

By following these practices and building your catalog incrementally as demonstrated in this chapter, you'll create a data catalog that serves both current users and future needs effectively.

Chapter 4. Data Quality (DQV)

4.1. 2.1 Theory and Concepts

Data quality represents the cornerstone of trustworthy data ecosystems. While the previous chapter explored how to describe datasets, this chapter addresses the critical question of how good that data actually is. Creators and publishers of data often have a pretty good understanding of the quality of their data, but this knowledge usually does not leave the team/organizational boundaries. This makes it much more difficult for (potential) users of the data to determine whether the data meets their quality needs.

Publishing data quality data as part of the data catalog is a good way to give consumers a sense of what to expect. This can be a start of a conversation between publishers and consumers when the data quality falls short for new usecases.

Finally, some publishers feel some aprehension about being open about data quality and prefer to keep this information confidential. While this is understandable, and in some cases unavoidable, consumers of data will experience this data quality, whether you share it or not. They will often design systems to evaluate the quality of the incoming data for them selves. Being open about it up front tends to lead to better outcomes and more constructive interactions with the users of your data. So this is highly recommonded.

4.1.1. What is DQV?

The Data Quality Vocabulary (DQV) provides a standardized framework for communicating data quality information in a way that enables data consumers to make informed decisions about data usage.

DQV is a W3C Recommendation that defines a set of classes and properties for representing data quality information. DQV enables data publishers to express quality assessments, measurements, and annotations in a standardized format that can be understood and processed by both humans and machines. The vocabulary supports a wide range of quality assessment approaches, from automated metrics to human evaluations, from single measurements to comprehensive quality certificates.

Just as with the other standards referenced in this work, the DQV model can also be seen as an anatomy of data quality. It helps us think about what we might need to know about data quality and in doing so, helps us think about and scope the actions we should take to obtain and share this information.

DQV provides concepts and terminology to addresses the fundamental challenge of data quality communication: how do we express that data is good or bad, complete or incomplete, accurate or inaccurate, in a way that is meaningful, comparable, and (idally) actionable? The vocabulary provides the semantic foundation for answering these questions while maintaining flexibility to accommodate diverse quality assessment methodologies and domain-specific requirements.

4.1.2. Core DQV Concepts

DQV is built around a framework that organizes quality information into meaningful categories. At the foundation are **Metrics**, which define specific measurable aspects of data quality such as completeness, accuracy, or availability. These metrics are applied to datasets through **QualityMeasurements**, which capture the actual observed values and the context in which they were measured. When adding these metrics to your data catalog, you facilitate reusing them (where appropriate) across datasets/services.

The metrics themselves are organized into **Dimensions**, which group related metrics into broader quality categories. Common dimensions include completeness (how much data is missing), accuracy (how correct the data is), availability (whether the data can be accessed), and timeliness (how current the data is). These dimensions can be further organized into **Categories** that represent high-level quality frameworks such as ISO 25012 or domain-specific quality models.

```
classDiagram
    class QualityMeasurement {
        +identifier: string
        +computedOn: Resource
        +isMeasurementOf: Metric
        +value: float
        +generatedAtTime: date
    }

    class Metric {
        +identifier: string
        +definition: string
        +prefLabel: string
        +expectedDataType: Any
        +inDimension: string
    }

    class Resource

    class Dataset

    QualityMeasurement "1" -- "1" Metric : isMeasurementOf
    QualityMeasurement "1" -- "1" Resource : computedOn
    Dataset --|> Resource : inherits
```

This structure enables both detailed quality assessments and high-level quality summaries. Data consumers can drill down to specific metric values for detailed analysis, or examine dimension-level summaries for quick quality assessments. The framework also supports quality annotations that capture user feedback, expert evaluations, and quality certificates that provide formal attestations of data quality.

4.1.3. Quality Dimensions and Their Importance

Different applications and domains prioritize different quality dimensions. Understanding these

dimensions helps data publishers focus their quality assessment efforts on the aspects that matter most to their users. Annotating matrics with the dimensions they try to measure also helps others understand their meaning and purpose.

Completeness represents the idea that all data that is intended to be captured is also captured. Incomplete data can lead to biased analyses and incorrect conclusions. For example, a dataset with missing demographic information may not be suitable for population studies.

Accuracy is the idea that data should represent reality. Inaccurate data can undermine decision making and lead to costly errors. GPS coordinates with systematic bias, for instance, could misguide navigation systems. The challenge with this particular quality dimension is that while it is by many considered the most important, if not the ultimate, qualuty metric, it is also the hardest to evaluate (in a scalable/affordable matter). One effectively needs to have a witness: someone who goes out into the real world and verifies that what the data claims is in fact so.

Consistency represents the idea that data should not contradict itself. Intuitively: if data contradicts itself, something is wrong. This makes it a good proxy for accuracy: if data is consistent, it doesn't automatically mean it is accurate, but if it is inconsistent, we know something is innacurate. This can help us find inaccuracies and investigate them further.

Availability represents the idea that data shouldnt just exist, it should also be available to those who (legitimately) use it. Even high-quality data is useless if it cannot be retrieved. This dimension includes both technical accessibility and licensing accessibility.

Timeliness represents the idea that data has an expiration date. Data makes statements about reality, and the facts on the ground can change. When data is used when they are no longer accurate, the outcomes of that use may also be innacurate. For instance, Financial market data from yesterday may be too old for trading decisions, while historical climate data from decades ago may be perfectly suitable for climate change studies.

4.1.4. DQV Integration with DCAT

DQV builds upon the foundations established in previous chapters. While DCAT (Chapter 1) describes what datasets are available and how to access them, DQV tells us how good those datasets are for their intended purposes. This integration creates a comprehensive view of data resources that includes descriptive metadata and quality assessments. Provenance information will be covered in Chapter 3.

The integration works through quality annotations that link to DCAT datasets and distributions. Each quality measurement is associated with a specific dataset or distribution, creating a clear connection between quality information and the data resource it describes. This enables data consumers to evaluate quality alongside other metadata when making decisions about data usage.

PROV provenance information enhances quality assessments by providing context for quality measurements. When a dataset is derived from high-quality source data, consumers may have greater confidence in its quality. Conversely, datasets derived from low-quality sources may require additional scrutiny. The combination of provenance and quality information creates a complete picture of data trustworthiness.

4.2. 2.2 DQV in Simple Data Catalog

Simple Data Catalog implements a practical subset of DQV that focuses on the most commonly needed quality assessment capabilities. The implementation provides a straightforward interface for defining metrics, capturing measurements, and organizing quality information while maintaining compatibility with the DQV standard.

The framework supports both automated quality assessments and manual quality evaluations. Automated assessments can be captured through programmatic metric calculations, while manual evaluations can be documented through quality annotations and expert assessments. This flexibility accommodates diverse quality assessment workflows and organizational capabilities.

4.2.1. Quality Measurement Framework

The simple_data_catalog_model provides three main classes for quality information: `Metric`, `QualityMeasurement`, and quality annotations on datasets. The `Metric` class defines specific quality aspects that can be measured, including their definitions, expected data types, and organizational categorization. The `QualityMeasurement` class captures actual observations of those metrics applied to specific datasets.

```
# Example metric definition
metrics:
  - id: completeness-ratio
    definition: "Ratio of non-null values to total expected values"
    prefLabel: "Completeness Ratio"
    expectedDataType: "xsd:double"
    inDimension: "completeness"
```

The framework supports multiple data types for metric values, including boolean values for pass/fail assessments, numeric values for continuous measurements, and string values for categorical assessments. This flexibility enables diverse quality assessment approaches while maintaining standardized representation.

4.2.2. Metric Definitions and Calculations

Metrics in Simple Data Catalog can be defined at multiple levels of specificity. Some metrics may be generic and applicable across domains, such as availability or completeness ratios. Other metrics may be domain-specific, such as spatial accuracy for geographic data or temporal consistency for time-series data.

The framework supports both simple metrics that capture single values and complex metrics that may involve multiple calculations or sub-metrics. For example, a data quality index might combine several individual metrics into a single composite score. These complex metrics can be documented through their definitions and relationships to simpler metrics.

```
# Complex metric example
metrics:
```

```
  - id: overall-quality-index
    definition: "Weighted combination of completeness, accuracy, and availability
metrics"
    prefLabel: "Overall Quality Index"
    expectedDataType: "xsd:double"
    inDimension: "overall-quality"
    # Calculation method documented in description
```

4.2.3. Quality Reporting Structures

Quality measurements are organized to support both detailed analysis and summary reporting. Individual measurements can be aggregated at the dimension level to provide quality overviews, or examined in detail for specific quality issues. The framework supports trend analysis by capturing measurement timestamps, enabling quality monitoring over time.

The reporting structure also supports quality certificates and compliance attestations. These formal quality statements can be documented alongside metric measurements, providing both quantitative assessments and qualitative evaluations. This combination supports comprehensive quality communication that addresses both technical quality requirements and business quality expectations.

4.3. 2.3 Practical Examples

4.3.1. 2.3.1 Basic Quality Measurements

The simplest quality assessment scenario involves measuring a single metric for a dataset. This example demonstrates the fundamental DQV concepts in action, showing how to document basic quality information that helps users evaluate dataset fitness for purpose.

```
# Basic quality measurement example
metrics:
  - id: availability-check
    definition: "Binary check if dataset is accessible via its distributions"
    prefLabel: "Availability Check"
    expectedDataType: "xsd:boolean"
    inDimension: "availability"

qualityMeasurements:
  - id: availability-measurement-2024-01-15
    computedOn: temperature-readings
    isMeasurementOf: availability-check
    value: true
    generatedAtTime: "2024-01-15T10:30:00Z"

datasets:
  - id: temperature-readings
    title: "Daily Temperature Readings"
    description: "Temperature measurements collected from weather station"
```

```
    keywords:
      - temperature
      - climate
      - weather
    publisher:
      name: "Weather Service"
      email: "data@weather.gov"
    issued: "2024-01-15"
    modified: "2024-01-20"
    distributions:
      - id: csv-data
        title: "CSV format"
        description: "Temperature data in CSV format"
        downloadURL: "https://data.gov.climate/temperature.csv"
        mediaType: "text/csv"
        byteSize: "15000000"
```

This example shows a basic availability measurement for the temperature readings dataset. The metric defines what is being measured (availability), and the quality measurement captures the actual observation (true, indicating the dataset is accessible). The measurement includes a timestamp, enabling quality tracking over time.

4.3.2. 2.3.2 Multiple Quality Metrics

Real-world quality assessments typically involve multiple metrics across different quality dimensions. This example demonstrates how to capture comprehensive quality information that provides a complete picture of dataset quality.

```
# Multiple quality metrics example
metrics:
  - id: completeness-ratio
    definition: "Ratio of non-null values to total expected values"
    prefLabel: "Completeness Ratio"
    expectedDataType: "xsd:double"
    inDimension: "completeness"

  - id: accuracy-percentage
    definition: "Percentage of values that pass accuracy validation"
    prefLabel: "Accuracy Percentage"
    expectedDataType: "xsd:double"
    inDimension: "accuracy"

  - id: timeliness-score
    definition: "Score based on data recency relative to expected update frequency"
    prefLabel: "Timeliness Score"
    expectedDataType: "xsd:double"
    inDimension: "timeliness"

qualityMeasurements:
```

```yaml
  - id: completeness-measurement-2024-01-15
    computedOn: air-quality-data
    isMeasurementOf: completeness-ratio
    value: 0.95
    generatedAtTime: "2024-01-15T10:30:00Z"

  - id: accuracy-measurement-2024-01-15
    computedOn: air-quality-data
    isMeasurementOf: accuracy-percentage
    value: 0.87
    generatedAtTime: "2024-01-15T10:30:00Z"

  - id: timeliness-measurement-2024-01-15
    computedOn: air-quality-data
    isMeasurementOf: timeliness-score
    value: 0.92
    generatedAtTime: "2024-01-15T10:30:00Z"

datasets:
  - id: air-quality-data
    title: "Urban Air Quality Measurements"
    description: "Air quality index measurements from urban monitoring stations"
    keywords:
      - air-quality
      - pollution
      - urban
    publisher:
      name: "Environmental Protection Agency"
      email: "data@epa.gov"
    issued: "2024-01-15"
    modified: "2024-01-20"
    wasDerivedFrom:
      - raw-sensor-readings
```

This example shows comprehensive quality assessment for an air quality dataset, measuring three different quality dimensions. The completeness ratio of 0.95 indicates that 95% of expected data values are present, the accuracy percentage of 0.87 shows that 87% of values pass accuracy validation, and the timeliness score of 0.92 indicates relatively current data. Together, these measurements provide a complete quality picture that helps users evaluate the dataset for their specific needs.

4.3.3. 2.3.3 Quality Dimensions and Categories

Organizing metrics into dimensions and categories helps users understand quality assessments in context. This example demonstrates how to structure quality information using standard quality frameworks.

```yaml
# Quality dimensions and categories example
metrics:
```

```yaml
  # Completeness dimension
  - id: record-completeness
    definition: "Percentage of complete records in the dataset"
    prefLabel: "Record Completeness"
    expectedDataType: "xsd:double"
    inDimension: "completeness"

  - id: field-completeness
    definition: "Percentage of non-null values across all fields"
    prefLabel: "Field Completeness"
    expectedDataType: "xsd:double"
    inDimension: "completeness"

  # Accuracy dimension
  - id: format-accuracy
    definition: "Percentage of values in correct format"
    prefLabel: "Format Accuracy"
    expectedDataType: "xsd:double"
    inDimension: "accuracy"

  - id: value-accuracy
    definition: "Percentage of values within expected ranges"
    prefLabel: "Value Accuracy"
    expectedDataType: "xsd:double"
    inDimension: "accuracy"

  # Consistency dimension
  - id: internal-consistency
    definition: "Percentage of records without internal contradictions"
    prefLabel: "Internal Consistency"
    expectedDataType: "xsd:double"
    inDimension: "consistency"

qualityMeasurements:
  - id: record-completeness-measurement
    computedOn: population-census
    isMeasurementOf: record-completeness
    value: 0.98
    generatedAtTime: "2024-01-15T10:30:00Z"

  - id: field-completeness-measurement
    computedOn: population-census
    isMeasurementOf: field-completeness
    value: 0.94
    generatedAtTime: "2024-01-15T10:30:00Z"

  - id: format-accuracy-measurement
    computedOn: population-census
    isMeasurementOf: format-accuracy
    value: 0.99
    generatedAtTime: "2024-01-15T10:30:00Z"
```

```yaml
  - id: value-accuracy-measurement
    computedOn: population-census
    isMeasurementOf: value-accuracy
    value: 0.96
    generatedAtTime: "2024-01-15T10:30:00Z"

  - id: internal-consistency-measurement
    computedOn: population-census
    isMeasurementOf: internal-consistency
    value: 0.97
    generatedAtTime: "2024-01-15T10:30:00Z"

datasets:
  - id: population-census
    title: "Annual Population Census"
    description: "Demographic data collected through annual census surveys"
    keywords:
      - demographics
      - population
      - census
    publisher:
      name: "National Statistics Office"
      email: "census@stats.gov"
    issued: "2024-01-15"
    modified: "2024-01-20"
```

This example demonstrates quality assessment organized across three dimensions: completeness, accuracy, and consistency. Each dimension includes multiple specific metrics that provide detailed quality information. The dimension-level organization helps users understand quality patterns and identify areas that may need attention or improvement.

4.3.4. 2.3.4 Quality Trend Analysis

Quality often changes over time as datasets are updated, processed, or maintained. Tracking quality trends helps users understand data reliability and identify potential quality issues. This example shows how to capture quality measurements over time to support trend analysis.

```yaml
# Quality trend analysis example
qualityMeasurements:
  # January measurements
  - id: completeness-jan-2024
    computedOn: climate-data
    isMeasurementOf: completeness-ratio
    value: 0.92
    generatedAtTime: "2024-01-31T23:59:59Z"

  - id: accuracy-jan-2024
    computedOn: climate-data
```

```
    isMeasurementOf: accuracy-percentage
    value: 0.89
    generatedAtTime: "2024-01-31T23:59:59Z"

  # February measurements
  - id: completeness-feb-2024
    computedOn: climate-data
    isMeasurementOf: completeness-ratio
    value: 0.94
    generatedAtTime: "2024-02-29T23:59:59Z"

  - id: accuracy-feb-2024
    computedOn: climate-data
    isMeasurementOf: accuracy-percentage
    value: 0.91
    generatedAtTime: "2024-02-29T23:59:59Z"

  # March measurements
  - id: completeness-mar-2024
    computedOn: climate-data
    isMeasurementOf: completeness-ratio
    value: 0.96
    generatedAtTime: "2024-03-31T23:59:59Z"

  - id: accuracy-mar-2024
    computedOn: climate-data
    isMeasurementOf: accuracy-percentage
    value: 0.93
    generatedAtTime: "2024-03-31T23:59:59Z"

datasets:
  - id: climate-data
    title: "Monthly Climate Data"
    description: "Climate measurements updated monthly"
    keywords:
      - climate
      - weather
      - environmental
    publisher:
      name: "Climate Data Center"
      email: "data@climate.gov"
    issued: "2024-01-01"
    modified: "2024-03-31"
```

This example shows quality measurements captured over three months, revealing improvement trends in both completeness (from 0.92 to 0.96) and accuracy (from 0.89 to 0.93). Such trend analysis helps users understand data reliability and can inform decisions about when to use the data or when additional quality validation may be needed.

```
graph LR
```

```
    %% Quality Trend Analysis

    subgraph Completeness
        C1[Jan: 0.92] --> C2[Feb: 0.94] --> C3[Mar: 0.96]
    end

    subgraph Accuracy
        A1[Jan: 0.89] --> A2[Feb: 0.91] --> A3[Mar: 0.93]
    end

    style C1 fill:#e8f5e8
    style C2 fill:#c8e6c9
    style C3 fill:#a5d6a7
    style A1 fill:#e1f5fe
    style A2 fill:#b3e5fc
    style A3 fill:#81d4fa
```

4.3.5. 2.3.5 Integration with Provenance

Quality assessments become more meaningful when combined with provenance information. This example shows how quality measurements can be interpreted in the context of data derivation chains, helping users understand how quality evolves through data processing workflows.

```yaml
# Quality with provenance integration example
qualityMeasurements:
  - id: raw-data-quality
    computedOn: raw-sensor-data
    isMeasurementOf: completeness-ratio
    value: 0.85
    generatedAtTime: "2024-01-15T10:30:00Z"

  - id: cleaned-data-quality
    computedOn: cleaned-sensor-data
    isMeasurementOf: completeness-ratio
    value: 0.92
    generatedAtTime: "2024-01-15T11:45:00Z"

  - id: aggregated-data-quality
    computedOn: aggregated-climate-data
    isMeasurementOf: completeness-ratio
    value: 0.96
    generatedAtTime: "2024-01-15T14:20:00Z"

datasets:
  - id: raw-sensor-data
    title: "Raw Weather Station Sensor Data"
    description: "Direct readings from weather station sensors"
    publisher:
      name: "Weather Station Network"
```

```
      email: "data@stations.weather.gov"

  - id: cleaned-sensor-data
    title: "Cleaned Weather Station Data"
    description: "Sensor data after automated cleaning and validation"
    publisher:
      name: "Weather Data Processing Center"
      email: "processing@weather.gov"
    wasDerivedFrom:
      - raw-sensor-data

  - id: aggregated-climate-data
    title: "Monthly Aggregated Climate Data"
    description: "Climate data aggregated to monthly resolution"
    publisher:
      name: "Climate Data Center"
      email: "data@climate.gov"
    wasDerivedFrom:
      - cleaned-sensor-data
```

This example demonstrates how quality improves through data processing steps. The raw sensor data has 85% completeness, which improves to 92% after cleaning and validation, and reaches 96% after aggregation. This quality progression, documented alongside the provenance chain, helps users understand how data processing affects quality and builds trust in the final dataset.

```
graph TD
    A[Raw Sensor Data<br/>Completeness: 85%] -->|wasDerivedFrom| B[Cleaned Sensor
Data<br/>Completeness: 92%]
    B -->|wasDerivedFrom| C[Aggregated Climate Data<br/>Completeness: 96%]

    style A fill:#ffcdd2
    style B fill:#c8e6c9
    style C fill:#a5d6a7
```

4.3.6. 2.3.6 Quality Certificates and Compliance

Formal quality assessments often include certificates or compliance attestations that provide authoritative statements about data quality. This example shows how to document such formal quality evaluations alongside metric measurements.

```
# Quality certificate example
qualityMeasurements:
  - id: iso-compliance-measurement
    computedOn: certified-dataset
    isMeasurementOf: iso-25012-compliance
    value: true
    generatedAtTime: "2024-01-15T10:30:00Z"
```

```yaml
  - id: overall-quality-score
    computedOn: certified-dataset
    isMeasurementOf: quality-index
    value: 0.94
    generatedAtTime: "2024-01-15T10:30:00Z"

datasets:
  - id: certified-dataset
    title: "ISO 25012 Certified Dataset"
    description: "Dataset certified as compliant with ISO 25012 data quality
standards"
    keywords:
      - certified
      - iso-25012
      - high-quality
    publisher:
      name: "Quality Certified Data Provider"
      email: "data@quality-certified.org"
    license: "https://creativecommons.org/licenses/by/4.0/"
    # Quality certificate information in description
    conformsTo:
      - "https://www.iso.org/standard/57052.html"  # ISO 25012
    hasQualityAnnotation: true
```

This example shows a dataset with formal quality certification. The ISO 25012 compliance measurement indicates that the dataset meets international data quality standards, while the quality index provides a quantitative assessment. Such formal quality information helps users trust the data for critical applications and supports regulatory compliance requirements.

4.4. Best Practices

Effective data quality assessment requires attention to both technical and organizational considerations. The following practices help ensure that quality information is meaningful, actionable, and trustworthy.

Define Clear Quality Metrics: Always provide clear definitions for quality metrics that explain exactly what is being measured and how. Ambiguous metric definitions can lead to misinterpretation and incorrect quality assessments.

Use Standard Quality Frameworks: When possible, organize metrics using established quality frameworks like ISO 25012. This provides context for quality assessments and enables comparison across datasets and organizations.

Document Measurement Methods: Include information about how quality measurements were obtained, including whether they were automated calculations, manual assessments, or expert evaluations. This context helps users trust the quality information.

Capture Quality Timestamps: Always include timestamps for quality measurements to support trend analysis and ensure that quality information remains current. Outdated quality

measurements can mislead users about current data quality.

Balance Detail and Usability: Provide comprehensive quality information but organize it in ways that support both detailed analysis and quick assessments. Dimension-level summaries help users quickly evaluate quality while detailed metrics support in-depth analysis.

Integrate Quality with Provenance: Connect quality assessments with provenance information to show how quality evolves through data processing workflows. This integration helps users understand the context of quality measurements.

Update Quality Regularly: Establish processes for regular quality assessment and update quality measurements as datasets change. Current quality information is essential for maintaining user trust.

Consider User Perspectives: Assess quality from the perspective of intended users and use cases. Quality requirements vary significantly across applications, and quality assessments should reflect the needs of target users.

Document Quality Limitations: Be transparent about quality limitations and known issues. Honest communication about quality problems builds trust and helps users make informed decisions.

Support Quality Improvement: Use quality assessments not just for evaluation but also for identifying improvement opportunities. Quality measurements should guide data quality enhancement efforts.

Chapter 5. Data Lineage (PROV)

5.1. 3.1 Theory and Concepts

Data lineage tells the story of how datasets come into existence, how they transform over time, and how they relate to one another through derivation chains. The Provenance Ontology (PROV-O) provides a standardized framework for capturing this narrative, enabling data consumers to understand the origins and transformations of datasets.

5.1.1. What is PROV?

The Provenance Ontology (PROV-O) is a W3C Recommendation that defines a set of classes and properties for representing and exchanging provenance information. PROV captures the relationships between entities, activities, and agents that participate in the creation and evolution of data. PROV answers the fundamental questions about data: where did it come from, how was it transformed, and what is its relationship to other data?

5.1.2. Core PROV Concepts

PROV is built around three fundamental concepts that work together to create a complete provenance story. **Entities** represent the data artifacts themselves - datasets, distributions, or any data-bearing resources. **Activities** describe the processes that transform these artifacts, though these are not currently implemented in the simple_data_catalog_model. **Agents** are the entities responsible for carrying out those activities, also not yet implemented in our model.

```
graph TD
    A[Entity] -->|wasDerivedFrom| A[Entity]
    D[Activity] -->|used| A
    A -->|wasGeneratedBy| D
    A -->|wasAttributedTo| C[Agent]
    C -->|actedOnBehalfOf| C
    D -->|wasAssociatedWith| C


    style A fill:#e8f5e8
    style C fill:#e1f5fe
    style D fill:#e1f5fe
```

The current simple_data_catalog_model implements a subset of PROV concepts, focusing on entity-to-entity relationships. The `wasDerivedFrom` relationship captures how one dataset relied on another, even when the specific transformation activity is unknown. Think of this as references and the bibliography in scientific literature. This forms the foundation of provenance tracking in our data catalog, enabling the reconstruction of data processing histories.

5.1.3. Provenance for Data Trust

The importance of provenance for data trust cannot be overstated. In data-driven decision making,

understanding the origins and transformations of data is essential for assessing reliability and fitness for purpose. Provenance information enables data consumers to evaluate data quality, identify potential biases, and make informed decisions about data usage. It also supports regulatory compliance, scientific reproducibility, and data governance requirements.

5.1.4. PROV Integration with DCAT

PROV can be combined DCAT to provide comprehensive metadata about datasets. While DCAT describes what datasets are available and how to access them, PROV explains where those datasets came from and how they relate to other datasets. This integration creates a complete picture of data resources, combining descriptive metadata with provenance information. In our implementation, this means every dataset can optionally include provenance relationships that trace its derivation from other datasets in the catalog.

In the previous chapter, we explored how to create datasets with rich descriptive metadata. Now we extend those datasets with provenance information that shows their place in the broader data ecosystem. A dataset that describes temperature readings might be derived from raw sensor data, which itself might be derived from weather station logs. This derivation chain helps users understand the processing history and assess data quality.

5.2. 3.2 PROV in Simple Data Catalog

Simple Data Catalog implements a focused subset of PROV. The current implementation prioritizes practical usability while maintaining compatibility with the PROV-O standard. The framework provides a straightforward interface for defining derivation relationships between datasets, making provenance capture accessible without requiring deep expertise in semantic web technologies.

The current implementation includes one primary PROV property: `wasDerivedFrom`, which links datasets to their source datasets. This property follows the PROV-O specification exactly, using the standard namespace and semantics. The mapping preserves the semantic meaning of PROV while providing a user-friendly YAML format that integrates seamlessly with existing dataset metadata.

The implementation focuses on the most common provenance use case: tracking how datasets are derived from other datasets. This approach covers the majority of real-world scenarios where data consumers need to understand data origins, while keeping the complexity manageable. Future versions of the data model may expand to include full PROV support with activities and agents, but the current implementation provides a solid foundation for provenance tracking.

5.2.1. Current PROV Support

YAML Element	PROV Property	Description
`wasDerivedFrom`	`prov:wasDerivedFrom`	Links a dataset to its source datasets
`generatedAtTime`	`prov:generatedAtTime`	Timestamp when an entity was generated (available in QualityMeasurement)

5.2.2. Integration with DCAT

The PROV properties extend the DCAT dataset definitions from Chapter 1. Every dataset can optionally include provenance information that traces its derivation chain. This provenance information is stored alongside the descriptive metadata, creating a comprehensive view of each dataset that includes both what it contains and where it came from.

5.2.3. 2.2.1 Lineage Visualization and Supply Chain Analysis

The simple_data_catalog integrates with the simple-data-catalog-generator library to automatically generate lineage visualizations and supply chain analysis from PROV relationships. These visualizations help users understand data dependencies and assess data quality across the entire data supply chain.

Lineage Diagrams

When datasets include `wasDerivedFrom` relationships, the generator creates Mermaid flowchart diagrams that visualize the complete data lineage. These diagrams show how datasets are connected through derivation relationships, making it easy to trace data transformations and identify upstream dependencies.

```
graph TD
    B[cleaned-sensor-data] --> A[raw-sensor-data]
    C[aggregated-climate-data] --> B[cleaned-sensor-data]
    D[climate-trends-analysis] --> C[aggregated-climate-data]
    F[climate-index] --> E[temperature-series]
    F[climate-index] --> G[precipitation-series]
    D[climate-trends-analysis] --> F[climate-index]
```

The lineage diagrams are automatically generated from the YAML provenance relationships and displayed on dataset detail pages. Each node represents a dataset, and arrows indicate derivation relationships following the `wasDerivedFrom` property.

Supply Chain Analysis

The generator also performs supply chain analysis by examining upstream data quality annotations. This analysis helps users understand the quality landscape of their data dependencies by aggregating quality measurements from all source datasets.

```
pie title Input Datasets Quality Measurements
    "with quality measurements" : 3
    "without quality measurements" : 2
```

The supply chain analysis provides insights into **Quality Coverage**. It shows the percentage of upstream datasets that have quality measurements

Implementation Example

To enable lineage visualization and supply chain analysis, simply include `wasDerivedFrom` relationships in your dataset definitions:

```yaml
datasets:
  - id: climate-trends-analysis
    title: "Climate Trends Analysis Report"
    description: "Statistical analysis of long-term climate trends"
    wasDerivedFrom:
      - aggregated-climate-data
      - climate-index
    hasQualityAnnotation: true
    qualityMeasurements:
      - metric: completeness
        value: 0.95
        method: automated_check

  - id: aggregated-climate-data
    title: "Monthly Aggregated Climate Data"
    wasDerivedFrom:
      - cleaned-sensor-data
    hasQualityAnnotation: true
    qualityMeasurements:
      - metric: accuracy
        value: 0.88
        method: manual_validation

  - id: cleaned-sensor-data
    title: "Cleaned Weather Station Data"
    wasDerivedFrom:
      - raw-sensor-data
```

When processed by the simple-data-catalog-generator, this configuration will: 1. Generate a lineage diagram showing the derivation chain 2. Create a supply chain analysis showing that 2 out of 3 upstream datasets have quality measurements 3. Provide clickable navigation between related datasets

Benefits for Data Governance

The lineage visualization and supply chain analysis capabilities provide several benefits for data governance:

- **Transparency**: Clear visualization of data dependencies and transformations
- **Quality Assessment**: Quick identification of data quality gaps in the supply chain
- **Impact Analysis**: Understanding how changes in upstream datasets affect downstream products
- **Compliance Support**: Documentation of data provenance for regulatory requirements

- **Trust Building**: Visual evidence of data processing and quality controls

These features make the simple_data_catalog particularly valuable for organizations that need to demonstrate data governance, track data quality across complex data ecosystems, and provide stakeholders with clear understanding of data origins and reliability.

5.3. 3.3 Practical Examples

5.3.1. 3.3.1 Basic Derivation Tracking

The simplest provenance scenario involves a dataset that is directly derived from a single source dataset. This example demonstrates the fundamental PROV concept in action, showing how to document the basic story of data transformation.

```yaml
# Basic derivation example
datasets:
  - id: temperature-readings
    title: "Daily Temperature Readings"
    description: "Temperature measurements collected from weather station"
    keywords:
      - temperature
      - climate
      - weather
    publisher:
      name: "Weather Service"
      email: "data@weather.gov"
    issued: "2024-01-15"
    modified: "2024-01-20"
    distributions:
      - id: csv-data
        title: "CSV format"
        description: "Temperature data in CSV format"
        downloadURL: "https://data.gov.climate/temperature.csv"
        mediaType: "text/csv"
        byteSize: "15000000"

  - id: processed-temperature-data
    title: "Quality-Controlled Temperature Data"
    description: "Temperature measurements after quality control processing"
    keywords:
      - temperature
      - climate
      - quality-controlled
    publisher:
      name: "Climate Data Center"
      email: "data@climate.gov"
    issued: "2024-01-21"
    modified: "2024-01-21"
    wasDerivedFrom:
```

```
      - temperature-readings
```

This example shows how processed temperature data is derived from raw temperature readings. The `wasDerivedFrom` property creates a clear link between the two datasets, enabling users to trace the data processing history. The derived dataset maintains its own metadata while explicitly referencing its source, creating a transparent provenance chain.

5.3.2. 3.3.2 Multi-Source Derivation

A common scenario involves datasets that are created by combining multiple source datasets. This example demonstrates how to document complex derivation relationships where a single dataset draws from multiple sources.

```yaml
# Multi-source derivation example
datasets:
  - id: temperature-series
    title: "Historical Temperature Series"
    description: "Monthly temperature measurements from 1990-2020"
    keywords:
      - temperature
      - historical
      - climate
    temporal:
      start: "1990-01-01"
      end: "2020-12-31"
    publisher:
      name: "National Weather Service"
      email: "data@weather.gov"

  - id: precipitation-series
    title: "Historical Precipitation Series"
    description: "Monthly precipitation measurements from 1990-2020"
    keywords:
      - precipitation
      - historical
      - climate
    temporal:
      start: "1990-01-01"
      end: "2020-12-31"
    publisher:
      name: "National Weather Service"
      email: "data@weather.gov"

  - id: climate-index
    title: "Combined Climate Index"
    description: "Climate index calculated from temperature and precipitation data"
    keywords:
      - climate-index
      - temperature
```

```
      - precipitation
      - combined
    temporal:
      start: "1990-01-01"
      end: "2020-12-31"
    publisher:
      name: "Climate Research Institute"
      email: "data@climate-institute.gov"
    wasDerivedFrom:
      - temperature-series
      - precipitation-series
```

This example shows how a climate index dataset is derived from both temperature and precipitation series. The multi-valued `wasDerivedFrom` property creates links to both source datasets, documenting the complete provenance of the derived dataset. Users can trace back to understand exactly what data sources contributed to the climate index calculation.

5.3.3. 3.3.3 Derivation Chains

Complex data processing workflows often create chains of derived datasets, where each step builds upon the previous one. This example demonstrates how to document multi-level derivation chains that show the complete data processing history.

```
# Derivation chain example
datasets:
  - id: raw-sensor-data
    title: "Raw Weather Station Sensor Data"
    description: "Direct readings from weather station sensors"
    keywords:
      - raw
      - sensor
      - weather
    publisher:
      name: "Weather Station Network"
      email: "data@stations.weather.gov"

  - id: cleaned-sensor-data
    title: "Cleaned Weather Station Data"
    description: "Sensor data after automated cleaning and validation"
    keywords:
      - cleaned
      - validated
      - weather
    publisher:
      name: "Weather Data Processing Center"
      email: "processing@weather.gov"
    wasDerivedFrom:
      - raw-sensor-data
```

```yaml
  - id: aggregated-climate-data
    title: "Monthly Aggregated Climate Data"
    description: "Climate data aggregated to monthly resolution"
    keywords:
      - aggregated
      - monthly
      - climate
    publisher:
      name: "Climate Data Center"
      email: "data@climate.gov"
    wasDerivedFrom:
      - cleaned-sensor-data

  - id: climate-trends-analysis
    title: "Climate Trends Analysis Report"
    description: "Statistical analysis of long-term climate trends"
    keywords:
      - trends
      - analysis
      - climate
    publisher:
      name: "Climate Research Institute"
      email: "research@climate-institute.gov"
    wasDerivedFrom:
      - aggregated-climate-data
```

This example demonstrates a complete derivation chain from raw sensor data to final climate trends analysis. Each dataset in the chain explicitly references its immediate predecessor, creating a clear trail of data transformations. Users can follow this chain backward to understand the complete processing history, or forward to see how the data evolves through each processing step.

5.3.4. 3.3.4 Integration with DCAT Metadata

Provenance information integrates seamlessly with the DCAT metadata structures from Chapter 1. This example shows how derivation relationships complement the descriptive metadata to create a comprehensive dataset description.

```yaml
# Complete dataset with DCAT metadata and PROV provenance
datasets:
  - id: national-air-quality-index
    title: "National Air Quality Index"
    description: "Daily air quality index calculated from multiple pollutant
measurements"
    keywords:
      - air-quality
      - pollution
      - environmental
      - health
    temporal:
```

```
      start: "2023-01-01"
      end: "2023-12-31"
    spatial:
      - "Country of Exampleland"
    accrualPeriodicity: "daily"
    publisher:
      name: "Environmental Protection Agency"
      email: "data@epa.gov"
      type: "organization"
    theme:
      - environment
      - health
      - air-quality
    license: "https://creativecommons.org/licenses/by/4.0/"
    wasDerivedFrom:
      - pm25-measurements
      - no2-measurements
      - o3-measurements
    distributions:
      - id: aqi-daily-csv
        title: "Daily AQI CSV Data"
        description: "Air quality index values in CSV format"
        downloadURL: "https://data.epa.gov/aqi/daily/2023.csv"
        mediaType: "text/csv"
        byteSize: "5000000"
      - id: aqi-api
        title: "AQI REST API"
        description: "Real-time air quality index API"
        accessURL: "https://api.epa.gov/aqi/v1"
        mediaType: "application/json"
        conformsTo: "https://api.epa.gov/docs/aqi/v1"

  - id: pm25-measurements
    title: "PM2.5 Concentration Measurements"
    description: "Particulate matter 2.5 micrometer measurements from monitoring
stations"
    keywords:
      - pm25
      - particulate-matter
      - pollution
    publisher:
      name: "Air Quality Monitoring Network"
      email: "data@air-quality.gov"

  - id: no2-measurements
    title: "NO2 Concentration Measurements"
    description: "Nitrogen dioxide measurements from monitoring stations"
    keywords:
      - no2
      - nitrogen-dioxide
      - pollution
```

```
    publisher:
      name: "Air Quality Monitoring Network"
      email: "data@air-quality.gov"

  - id: o3-measurements
    title: "Ozone Concentration Measurements"
    description: "Ground-level ozone measurements from monitoring stations"
    keywords:
      - ozone
      - o3
      - pollution
    publisher:
      name: "Air Quality Monitoring Network"
      email: "data@air-quality.gov"
```

This comprehensive example shows how provenance information enhances the rich DCAT metadata from Chapter 1. The national air quality index dataset includes complete descriptive metadata while also documenting its derivation from three pollutant measurement datasets. This integration creates a complete picture that tells users both what the dataset contains and where it came from.

5.3.5. 3.3.6 Cross-Referencing Provenance

When documenting provenance across multiple datasets, you may need to reference datasets that are defined in separate YAML files or in other catalogs. PROV supports this through the use of full dataset identifiers.

```
# Cross-referencing provenance example
datasets:
  - id: https://data.weather.gov/datasets/temperature-series
    title: "Historical Temperature Series"
    description: "Global temperature measurements from 1880-2020"
    publisher:
      name: "Global Climate Organization"
      email: "data@globalclimate.org"

  - id: local-climate-analysis
    title: "Regional Climate Impact Analysis"
    description: "Regional analysis based on global temperature data"
    publisher:
      name: "Regional Climate Center"
      email: "data@regional-climate.gov"
    wasDerivedFrom:
      - https://data.weather.gov/datasets/temperature-series
```

This example shows how to reference datasets in external catalogs using their full identifiers. This approach enables provenance tracking across organizational boundaries, supporting federated data catalogs and collaborative data ecosystems.

5.4. 3.4 Current Limitations and Future Enhancements

5.4.1. Current Implementation Limitations

The current simple_data_catalog_model implements a focused subset of PROV that prioritizes practical usability over comprehensive provenance tracking. This approach has several limitations that users should understand:

Missing PROV Concepts: The current implementation does not include: - **Activities**: Processing steps that transform data - **Agents**: People, organizations, or software responsible for activities - **Temporal properties**: `startedAtTime`, `endedAtTime` for activities - **Qualified relationships**: Detailed attribution, usage, or generation information - **Complex derivation types**: Specialization, revision, quotation relationships

Limited Provenance Depth: Current implementation only supports: - Simple entity-to-entity derivation relationships - Basic attribution through dataset publishers.

5.4.2. Future Enhancement Roadmap

The simple_data_catalog_model roadmap includes plans for progressive PROV enhancement:

- Import of full LinkML PROV schema
- Activity definitions with temporal properties
- Agent classes (Person, Organization, SoftwareAgent)

5.4.3. Migration Strategy

When future enhancements are implemented, existing YAML configurations will remain backward compatible. New properties will be added as optional elements, allowing gradual adoption of enhanced provenance features.

5.5. Best Practices

Effective provenance documentation with current capabilities requires attention to several key principles:

Document Derivations Clearly: Always include `wasDerivedFrom` relationships when datasets are created from other datasets. This creates essential provenance chains that users can follow to understand data origins.

Maintain Consistent Identifiers: Use stable, unique identifiers for all datasets that may be referenced in provenance relationships. Consider using persistent URLs or DOI-style identifiers for important datasets.

Balance Detail and Usability: While provenance information is valuable, avoid creating overly complex derivation chains that might confuse users. Focus on the most important source datasets that users would want to understand.

Update Provenance Regularly: When datasets are updated or reprocessed, update their `wasDerivedFrom` relationships to reflect current processing history. Outdated provenance information can mislead users about data quality and reliability.

Consider Cross-Organizational Needs: When working with data from multiple organizations, use full identifier URLs that can be resolved across domains. This supports federated provenance tracking and collaboration.

Document Processing Context: Even though activities are not currently supported in the data model, include relevant processing information in dataset descriptions or documentation to help users understand derivation context.

Chapter 6. Data Policy (ODRL)

6.1. 4.1 Theory and Concepts

Data governance represents framework through which organizations ensure that their data assets are managed responsibly, ethically, and in compliance with applicable regulations. While previous chapters addressed how to describe datasets, trace their origins, and assess their quality, this chapter tackles the equally important question of what can and cannot be done with that data. The Open Digital Rights Language (ODRL) provides a standardized vocabulary for expressing policies, permissions, obligations, and prohibitions that govern the use of data assets.

6.1.1. How Policy Shapes Data Use

Data policies serve as critical bridge between data availability and appropriate usage, transforming raw data assets into governed resources that can be shared with confidence. Policies shape data use by establishing clear boundaries and expectations that answer fundamental questions: Who can access this data? What can they do with it? Under what conditions? What obligations must they fulfill? What actions are explicitly prohibited?

Well-designed policies enable organizations to balance competing priorities: innovation versus control, accessibility versus security, sharing versus protection. Open data policies maximize accessibility by granting broad permissions for use, modification, and redistribution, encouraging innovation and collaboration. Restricted policies protect sensitive information by limiting access to specific parties, prohibiting certain actions like commercial use, and requiring safeguards like confidentiality agreements. Conditional policies enable nuanced sharing models where permissions are granted contingent upon fulfilling specific duties, such as attribution, compensation, or reciprocal data sharing.

The policy framework directly impacts how data flows through organizations and ecosystems. Permissive policies accelerate data-driven innovation by removing barriers to access and reuse. Restrictive policies protect privacy, intellectual property, and security interests while potentially limiting data utility. Conditional policies create trusted environments where sensitive data can be shared under carefully managed terms that ensure appropriate use and mutual benefit. By clearly defining these parameters, policies enable both human users and automated systems to understand and respect the intended use of data assets.

6.1.2. Defining Roles and Responsibilities

Beyond specifying what actions are permitted or prohibited, policies establish roles and responsibilities that govern data relationships between publishers and users. ODRL policies define clear accountability structures by identifying the parties involved and their respective obligations, creating a foundation for trusted data sharing relationships.

Data Publishers (also known as assigners in ODRL terminology) bear responsibility for establishing appropriate usage terms, maintaining data quality, providing accurate documentation, and ensuring compliance with applicable regulations. They must determine the appropriate policy framework based on data sensitivity, business requirements, legal obligations, and stakeholder

interests. Publishers are responsible for communicating policies clearly, making them discoverable through catalog metadata, and providing mechanisms for policy enforcement and compliance monitoring.

Data Users (assignees in ODRL) acquire specific rights and responsibilities based on the policies they accept. Their responsibilities include complying with usage restrictions, fulfilling attached duties such as attribution or compensation, maintaining data security, respecting privacy obligations, and reporting violations or issues. Users must understand the policies that govern their access to data and ensure their intended use aligns with granted permissions. In many ecosystems, users also contribute to governance by providing feedback on policy effectiveness and suggesting improvements.

Third Parties such as service providers, auditors, and regulators may also have defined roles within policy frameworks. Service providers might be granted limited permissions to process or analyze data on behalf of users, with specific obligations to maintain security and confidentiality. Auditors may have read-only access for compliance verification, with duties to report findings and maintain confidentiality. Regulators might have oversight permissions with responsibilities to ensure compliance with legal requirements.

This role-based approach to policy management creates clear accountability structures that support compliance and trust. When everyone understands their responsibilities and the consequences of policy violations, data sharing becomes more predictable and manageable. Policies become living agreements that evolve as relationships mature, regulations change, and data uses emerge, providing the flexibility needed for dynamic data ecosystems.

6.1.3. What is ODRL?

The Open Digital Rights Language (ODRL) is a W3C Recommendation that defines a flexible, interoperable framework for expressing digital rights and policies. ODRL enables organizations to specify usage rules, access conditions, and compliance requirements in a machine-readable format that can be interpreted and enforced by automated systems. The language supports a wide range of policy scenarios, from simple open data licenses to complex data sharing agreements with multiple parties and conditional permissions.

Just as with the other standards referenced in this work, the ODRL model can also be seen as an anatomy of data quality. It helps us think about what we might need to know about data policy and govcernance and in doing so, helps us think about and scope the actions we should take to obtain and share this information.

ODRL addresses the fundamental challenge of policy expression: how do we communicate what users can and cannot do with data, under what conditions, and with what obligations, in a way that is both legally sound and technically enforceable? The language provides the semantic foundation for answering these questions while accommodating diverse policy models including open licensing, restricted access, conditional use, and data sovereignty requirements.

6.1.4. Core ODRL Concepts

ODRL is built around a set of interconnected concepts that work together to create comprehensive policy frameworks. **Policies** serve as containers for rules and represent the highest level of

organization. A policy can be a simple set of rules or a complex agreement between multiple parties. Each policy must have a unique identifier and typically includes descriptive metadata to help humans understand its purpose and scope.

The rules within policies come in three fundamental types. **Permissions** grant users the right to perform specific actions, such as reading, copying, or redistributing data. **Prohibitions** explicitly forbid certain actions, such as commercial use or reverse engineering. **Duties** represent obligations that must be fulfilled, either as standalone requirements or as conditions attached to permissions. These rule types can be combined to create sophisticated policy frameworks that balance access with responsibility.

```
graph TD
    A[Policy] -->|permission| B[Permission]
    A -->|prohibition| C[Prohibition]
    A -->|obligation| D[Duty]
    B -->|action| E[Action]
    C -->|action| E[Action]
    D -->|action| E[Action]
    B -->|relation| F[Asset]
    C -->|relation| F[Asset]
    D -->|relation| F[Asset]
    B -->|function| G[Party]
    C -->|function| G[Party]
    D -->|function| G[Party]
    B -->|duty| D
    D -->|remedy| C
    D -->|consequence| D

    style A fill:#e8f5e8
    style B fill:#e1f5fe
    style C fill:#ffebee
    style D fill:#fff3e0
    style E fill:#f3e5f5
    style F fill:#e0f2f1
    style G fill:#e0f2f1
```

Assets represent the resources to which policies apply, such as datasets, distributions, or services. **Parties** represent the entities involved in policies, including assigners (who grant permissions), assignees (who receive permissions), and other stakeholders. **Actions** describe the specific activities that are permitted, prohibited, or required, such as read, write, distribute, or analyze.

Constraints add conditional logic to rules, specifying when, where, and how actions may be performed. These can include temporal constraints (time periods), spatial constraints (geographic boundaries), purpose limitations (specific use cases), and technical constraints (format requirements). Constraints enable the creation of fine-grained policies that adapt to different contexts and requirements.

6.1.5. Data Governance and Policy Management

Data governance encompasses the policies, standards, and practices that ensure data is managed effectively throughout its lifecycle. Effective data governance requires balancing competing needs: accessibility versus control, innovation versus compliance, sharing versus security. ODRL provides the technical foundation for expressing these governance decisions in a way that can be systematically applied and enforced.

The importance of robust data governance has grown significantly as organizations face increasing regulatory scrutiny, privacy requirements, and ethical obligations. Regulations such as GDPR, CCPA, and industry-specific compliance frameworks require organizations to demonstrate control over how data is used, who can access it, and for what purposes. ODRL policies serve as the mechanism for translating these legal and ethical requirements into enforceable rules that can be applied automatically.

Data governance also addresses the practical challenges of managing data in complex ecosystems with multiple stakeholders. Research institutions need to share data while protecting intellectual property. Government agencies must balance transparency with security concerns. Commercial organizations need to enable data monetization while respecting privacy and contractual obligations. ODRL provides the vocabulary for expressing these nuanced arrangements in a standardized, interoperable format.

6.1.6. ODRL Integration with DCAT and Other Standards

ODRL is designed to complement other data standards, creating a comprehensive ecosystem for data management. While DCAT describes what datasets are available and how to access them, PROV explains where they came from, and DQV assesses their quality, ODRL specifies what can be done with them. This integration enables organizations to create complete data catalog entries that include not just descriptive metadata but also usage policies.

In our implementation, ODRL policies are attached to datasets through the `hasPolicy` property, creating direct links between data resources and their governance frameworks. This integration enables automated policy enforcement systems to discover applicable policies simply by examining catalog metadata. It also supports policy discovery for human users, who can understand the terms of use before accessing or downloading data.

The integration extends to relationships between datasets. When a dataset is derived from another dataset, as described in Chapter 3, ODRL policies can specify whether the derived dataset inherits the original policies, what additional restrictions apply, and how attribution must be handled. This provenance-aware policy management ensures that usage terms are consistently applied throughout data lifecycles and derivation chains.

6.2. 4.2 ODRL in Simple Data Catalog

Simple Data Catalog implements ODRL through a focused set of classes and properties that support common policy scenarios while maintaining compatibility with the full ODRL specification. The implementation prioritizes practical usability, enabling data publishers to express meaningful policies without requiring deep expertise in rights expression languages or legal frameworks.

The core implementation includes the `Policy` class and its subclasses `Set`, which represent the organizational structure for rules. The model supports all three rule types: `Permission`, `Prohibition`, and `Duty`, each with appropriate properties for defining actions, targets, and constraints. The implementation follows the ODRL 2.2 specification exactly, using standard namespaces and semantics while providing a user-friendly YAML format that integrates seamlessly with existing catalog metadata.

6.2.1. Policy Structure and Implementation

Policies in Simple Data Catalog follow a hierarchical structure that mirrors the ODRL specification. Each policy has a unique identifier, optional title and description, and collections of rules. The YAML structure uses nested objects to represent these relationships, creating an intuitive mapping between the conceptual model and the implementation.

The model supports the full range of ODRL actions through string values, enabling expression of common data usage activities such as read, write, analyze, transform, and distribute. These actions can be combined with constraints to create sophisticated policy rules that adapt to different contexts. The implementation uses the standard ODRL action vocabulary while allowing extensions for domain-specific actions.

```yaml
policies:
  - uid: "https://example.com/policies/open-data"
    title: "Open Data License"
    description: "Permits unrestricted use, modification, and distribution"
    permission:
      - action: "use"
      - action: "reproduce"
      - action: "distribute"
      - action: "modify"
```

6.2.2. Party Management and Roles

The implementation includes a specialized enum for party collections based on the International Data Spaces Reference Architecture Model (IDS-RAM). This predefined set of roles simplifies policy specification by providing commonly used party types: Data Consumer, Data Provider, and Service Provider. These roles capture the typical relationships in data ecosystems while maintaining flexibility for more complex party definitions.

The party collection approach enables policies to target entire categories of users rather than specifying individual entities. This is particularly useful in data space scenarios where policies need to apply to all certified data consumers or all registered service providers. The enum can be extended to support additional roles or specific organizational structures.

6.2.3. Constraint Expressions and Evaluation

While the current model focuses on basic ODRL structures, it supports the expression of constraints through action specifications and party assignments. Future extensions may include more sophisticated constraint handling with temporal, spatial, and purpose limitations. The current

implementation provides a solid foundation for policy expression while maintaining simplicity for common use cases.

The constraint system is designed to integrate with the broader catalog ecosystem. Policies can reference datasets, distributions, and services directly through the model's relationship structure. This enables contextual policies that apply to specific resources while maintaining overall coherence within the catalog.

6.2.4. Policy Sets and Inheritance

ODRL supports the organization of policies into sets and inheritance of rules between policies. The Simple Data Catalog implementation includes the `Set` class as a default policy container and supports policy identification through unique identifiers. This structure enables the creation of policy hierarchies where general policies can be specialized or overridden by more specific ones.

Policy inheritance is particularly valuable in large catalogs with many datasets. A base policy can define standard terms of use for all datasets, with specialized policies for sensitive data, commercial data, or research datasets. This reduces policy duplication while maintaining flexibility for exceptions and special cases.

6.3. 4.3 Practical Examples

6.3.1. 4.3.1 Basic Permissions

The foundation of any policy system is the ability to grant permissions that enable data usage while establishing clear boundaries for acceptable use. The simplest scenario involves granting basic permissions for open data usage, where users can freely access, use, and share data without restrictions. This approach maximizes data accessibility while providing clear legal terms through an appropriate license.

A basic open data policy might specify permissions for common activities such as reading, analyzing, and redistributing data. The policy should clearly identify the target datasets and the parties to whom permissions apply. This creates a transparent environment where users understand exactly what they can do with the data without seeking additional permissions.

```yaml
# Basic open data policy
policies:
  - uid: "https://climate.org/policies/open-data"
    title: "Open Climate Data License"
    description: "Permits unrestricted use for any purpose"
    permission:
      - action: "use"
        description: "Permission to use data for any purpose"
      - action: "reproduce"
        description: "Permission to make copies of data"
      - action: "distribute"
        description: "Permission to share data with others"
```

This basic policy establishes a foundation for open data sharing while maintaining proper attribution and licensing. The action terms follow the ODRL standard vocabulary, ensuring interoperability with other systems that understand ODRL policies. The policy can be attached to climate datasets through the `hasPolicy` property, automatically applying these usage terms to all specified data assets.

6.3.2. 4.3.2 Intermediate: Permissions with Constraints

Real-world data sharing often requires more nuanced policies that include constraints and conditions. Research data, for example, might be freely available for academic use but restricted for commercial applications. Similarly, personal data might require specific safeguards regardless of the intended use. These intermediate policy scenarios demonstrate how constraints can be applied to create more targeted and context-aware permissions.

Consider a research dataset that should be available for academic purposes but protected from commercial exploitation. The policy would grant permissions for analysis and publication while including constraints on commercial use and redistribution. This type of policy is common in academic settings where data sharing is encouraged but intellectual property and commercial interests must be protected.

```
# Academic use policy with commercial restrictions
policies:
  - uid: "https://university.edu/policies/academic-use"
    title: "Academic Use Only License"
    description: "Permits academic use with commercial restrictions"
    permission:
      - action: "use"
        assignee: "Data Consumer"
        description: "Permission for academic research and educational use"
      - action: "analyze"
        assignee: "Data Consumer"
        description: "Permission to analyze data for research purposes"
    prohibition:
      - action: "commercialize"
        description: "Prohibition of commercial use of data"
      - action: "sell"
        description: "Prohibition of selling data or derived products"
```

This intermediate example introduces the concept of party targeting through the `assignee` property, which specifies that permissions apply to Data Consumers while prohibitions apply universally. The policy creates a balanced approach that enables research while protecting commercial interests. The use of both permissions and prohibitions demonstrates how ODRL can express complex policy requirements through complementary rule types.

The policy could be applied to research datasets where the university wants to support academic collaboration while maintaining control over commercial exploitation. This approach is particularly valuable for datasets that require significant investment to create and maintain, where some commercial protection is necessary to ensure sustainability.

6.3.3. 4.3.3 Advanced: Complex Permissions with Duties

The most sophisticated policy scenarios involve complex arrangements where permissions are conditional upon fulfilling specific duties or obligations. These scenarios often appear in data space environments, data sharing agreements between organizations, and contexts where data usage must be monitored, reported, or compensated. Advanced policies demonstrate the full expressive power of ODRL by combining permissions, prohibitions, and duties into comprehensive governance frameworks.

Consider a commercial data sharing agreement where organizations can access valuable data in exchange for providing usage analytics, maintaining confidentiality, and sharing derived improvements. The policy would grant permissions for commercial use while imposing duties that ensure fair exchange and mutual benefit. This type of arrangement is common in industry consortia, research collaborations, and data marketplaces.

```
# Commercial data sharing with reciprocal obligations
policies:
  - uid: "https://dataspace.org/policies/reciprocal-sharing"
    title: "Reciprocal Commercial Data Sharing"
    description: "Commercial use with reciprocal data sharing obligations"
    permission:
      - action: "use"
        assignee: "Data Consumer"
        description: "Permission for commercial use of data"
        duty:
          - action: "compensate"
            description: "Compensation based on usage volume"
          - action: "trackUsage"
            description: "Track and report data usage statistics"
      - action: "analyze"
        assignee: "Data Consumer"
        description: "Permission to analyze and derive insights"
        duty:
          - action: "shareImprovements"
            description: "Share derived data improvements with provider"
    duty:
      - action: "maintainConfidentiality"
        assignee: "Data Consumer"
        description: "Maintain confidentiality of sensitive information"
      - action: "reportViolations"
        assignee: "Data Consumer"
        description: "Report policy violations to data provider"
```

This advanced example demonstrates several sophisticated ODRL concepts. The permissions include attached duties that must be fulfilled to maintain the granted rights. The policy includes standalone duties that apply regardless of specific permissions. The combination creates a comprehensive framework for responsible data sharing that balances access with accountability.

The policy could be applied in a data marketplace scenario where organizations exchange valuable

datasets under carefully managed terms. The duties ensure fair compensation and reciprocal benefit, while prohibitions (which could be added separately) protect against misuse. This level of policy sophistication enables the creation of trusted data ecosystems where participants can share valuable assets with confidence that usage terms will be respected and enforced.

6.3.4. 4.3.4 Policy Management in Practice

Effective policy management requires not just creating individual policies but organizing them into coherent frameworks that can be systematically applied across datasets. This involves creating policy sets that address different use cases, managing policy inheritance and conflicts, and integrating policies with the broader catalog ecosystem. Proper policy management ensures consistency, reduces duplication, and maintains the overall integrity of the governance framework.

```yaml
# Policy set with multiple related policies
policies:
  - uid: "https://organization.org/policies/data-governance"
    title: "Comprehensive Data Governance Framework"
    description: "Complete set of policies for different data categories"

  - uid: "https://organization.org/policies/public-data"
    title: "Public Data Policy"
    description: "Open access for public domain data"
    permission:
      - action: "use"
      - action: "distribute"

  - uid: "https://organization.org/policies/internal-data"
    title: "Internal Use Only"
    description: "Restricted to internal organization use"
    permission:
      - action: "use"
        assignee: "Service Provider"
    prohibition:
      - action: "distribute"
      - action: "externalUse"
```

This example shows how multiple policies can be organized to address different data categories within the same organization. The public data policy enables broad access for datasets that can be freely shared, while the internal data policy restricts usage to approved parties. This tiered approach allows organizations to balance openness with security based on data sensitivity and business requirements.

When these policies are applied to datasets through the `hasPolicy` property, users can quickly understand the usage terms by examining the associated policies. Automated systems can enforce these rules consistently across different access points, ensuring compliance with organizational policies and regulatory requirements.

6.4. 4.4 Integration with Data Quality and Provenance

Policies become most powerful when integrated with other catalog metadata to create context-aware governance frameworks. The relationship between policies, quality, and provenance enables dynamic policy enforcement that adapts to data characteristics and usage history. This integration supports sophisticated governance scenarios where policy decisions depend on data quality assessments, provenance information, and real-time monitoring.

High-quality data might be eligible for more permissive policies, while lower-quality data might carry usage restrictions or quality improvement duties. Similarly, derived datasets might inherit policies from their source data or acquire additional obligations based on the transformations they undergo. This dynamic policy environment enables organizations to implement risk-appropriate governance that balances accessibility with data protection.

The integration also supports compliance monitoring and auditing. Policy violations can be correlated with quality issues or provenance gaps to identify systemic problems and improvement opportunities. Usage patterns can be analyzed to optimize policy terms and identify emerging needs for governance frameworks. This creates a feedback loop where policy management improves alongside data quality and provenance documentation.